

Step 4 — A Timeout and a Partial-Result Fallback — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (real `asyncio` code), **Advanced** (the real limitation of this approach, and why it's still the right call). Read Basic first even if you already know `asyncio` — the ordering of pieces here (thread, then timeout, then fallback) matters more than any one piece alone.

- [Hint 1 — Watching progress from outside the pipeline](#)
- [Hint 2 — The timeout wrapper, and what to return on failure](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — Watching Progress from Outside the Pipeline

Basic Version

`run_pipeline()` currently only returns its result at the very end. To build a fallback, `mcp_server.py` needs a way to see each round *as it happens*, not just at the end — because if the whole call times out, "at the end" never arrives.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Give `run_pipeline()` an optional callback argument, and call it right after each round is recorded — this is the same "let the caller plug in behavior" idea as a callback anywhere else in Python, nothing MCP-specific about it yet.

```
def run_pipeline(topic, max_rounds, progress_sink=None):
    ...
    for round_number in range(1, max_rounds + 1):
        ...
        round_info = RevisionRound(...)
        rounds.append(round_info)
        if progress_sink is not None:
            progress_sink(round_info)
    ...
```

In `mcp_server.py`, build a plain list and a small function that appends to it, then pass that function in as `progress_sink`:

```
progress_log = []

def record_round(round_info):
    progress_log.append(round_info)

result = run_pipeline(topic, config.max_revision_rounds, record_round)
```

Now `progress_log` has every round that finished, even if something after that point goes wrong.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Think about where `progress_sink` should live in `pipeline.py` — it needs to be called *after* a round is fully recorded (so `round_info` genuinely exists and is complete), and it needs to run even for a rejected round, not just an approved one, since a rejected round's summary is exactly what you'd want to fall back to if nothing ever gets approved.

Also think about `pipeline.py`'s error handling here, not just `mcp_server.py`'s. If `run_researcher` or `run_fact_checker` raises partway through a round, that round's `progress_sink` call never happens — which is correct, since the round never actually finished — but it means the fallback in `mcp_server.py` needs to be built around "whatever's in `progress_log` already," not "one round behind what actually happened." Wrap the two calls in a `try / except` inside the loop, and re-raise as a custom exception with enough detail (which round, what the underlying error was) that the fallback message can say something useful instead of just "it broke."

Difference between Basic, Intermediate, and Advanced: Basic names the problem — no visibility until the very end. Intermediate is the real callback pattern, in both files. Advanced is about getting the exact sequencing right: the callback only fires for a round that actually finished, and a mid-round failure needs its own clear error, not a silently missing round.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The Timeout Wrapper, and What to Return on Failure

Basic Version

`deep_research` needs to become an `async def` function. Inside it: run the pipeline with a time limit; if that limit is hit, or something breaks, return a clearly marked "not finished properly" answer built from whatever rounds already happened — never just hang, never return an empty string, never crash with no message.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

`run_pipeline()` is a plain, blocking (synchronous) function — it doesn't `await` anything, it just runs and takes however long it takes. You can't put a timeout directly on a synchronous call inside an `async def` the way you might expect; the whole event loop would just block for the same amount of time anyway. The fix is `asyncio.to_thread()`, which runs a synchronous function in a background thread and gives you back something you *can* `await` — and once you have that, `asyncio.wait_for()` can put a real time limit on it.

```
import asyncio

@mcp.tool()
async def deep_research(topic: str) -> str:
    progress_log = []

    def record_round(round_info):
        progress_log.append(round_info)

    try:
        result = await asyncio.wait_for(
            asyncio.to_thread(run_pipeline, topic, config.max_revision_rounds, re
            timeout=config.timeout_seconds,
        )
    except asyncio.TimeoutError:
        return format_partial_result(topic, progress_log, "Timed out.")
    except PipelineStepFailedError as exc:
        return format_partial_result(topic, progress_log, f"A pipeline step failed: {exc}")

    return format_result(result)
```

`format_partial_result` is a new function, separate from `format_result` — it builds a string from `progress_log` (whatever rounds happened) instead of from a completed `PipelineResult`, and it says clearly, right at the top, that this isn't a finished, confirmed answer.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Here's the part worth understanding before you consider this step done, not after:

`asyncio.wait_for()` stops your code from *waiting* on the background thread past the timeout. It does **not** stop the thread itself. `asyncio.to_thread()` hands the blocking call to a real OS thread, and Python has no clean, safe way to force-kill a thread mid-work. So when a timeout fires, `run_pipeline()` is still running in the background, orphaned — it'll keep calling `record_round()` on `progress_log` for a while after `deep_research` has already returned its fallback answer to the caller.

Is this a bug you need to fix for this step? No — it's a genuine, known limitation of this exact approach, and calling it out honestly is the point. Three things are worth knowing, even if you don't build all of them: (1) `progress_log` being appended to by an orphaned thread after you've already read from it is technically a race condition, though in CPython, list `.append()` is atomic enough that this won't corrupt the list — it just might grow after you've already used it, which is harmless here since you already returned. (2) The real fix for *true* cancellation is running the pipeline in a separate process (which actually can be killed) instead of a thread, or redesigning `run_pipeline()`'s loop to check a "should I stop?" flag between rounds — both are real production patterns, both are more machinery than this project needs. (3) Every extra `deep_research` call that times out leaves one more orphaned pipeline running in the background until it finishes or errors on its own — fine for a learning project processing one call at a time, a real concern for a server under real load, which is exactly the kind of thing a production readiness review would flag.

Say all of this out loud once, the same way the README asks — it's a real trade-off you're choosing, with your eyes open, not a gap you accidentally left.

Difference between Basic, Intermediate, and Advanced: Basic names what the async wrapper needs to do. Intermediate is the exact, correct `to_thread` + `wait_for` pattern, and the two exceptions to catch. Advanced is the honest limitation underneath that pattern — a timeout stops you from waiting, not the work itself — and why that's an acceptable, well-understood trade-off for this project rather than something to silently paper over.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)